LAPORAN PRAKTIKUM

POSTTEST (7)

ALGORITMA PEMROGRAMAN LANJUT

Disusun oleh:

Alya Kamila Fauziyah (2509106043)

Kelas (A2 ‘25)

PROGRAM STUDI INFORMATIKA

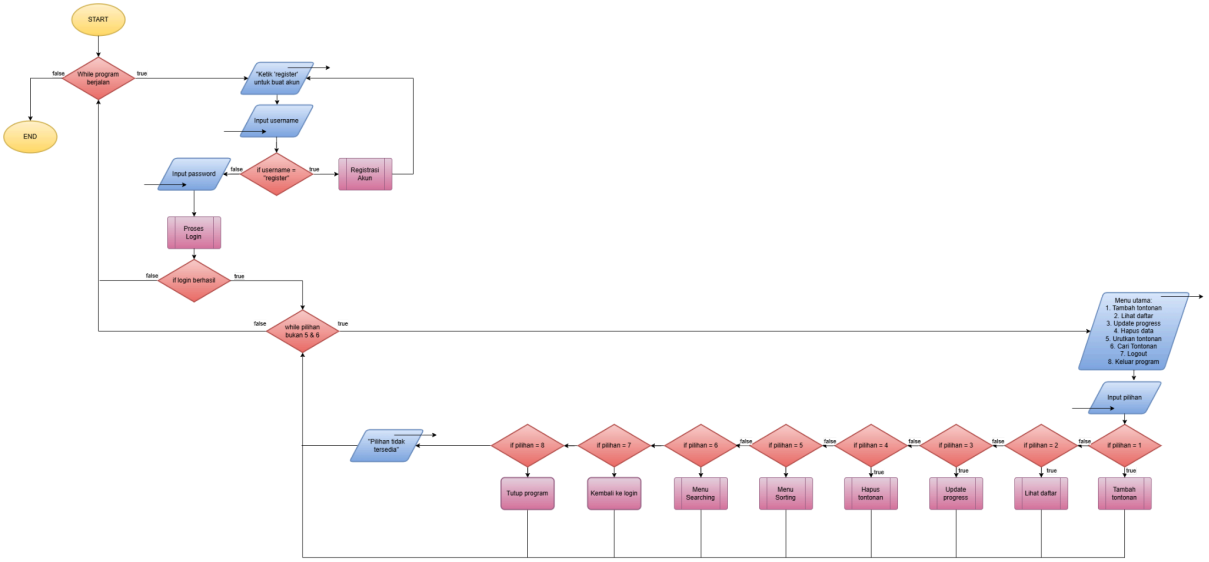
UNIVERSITAS MULAWARMAN

SAMARINDA

2025

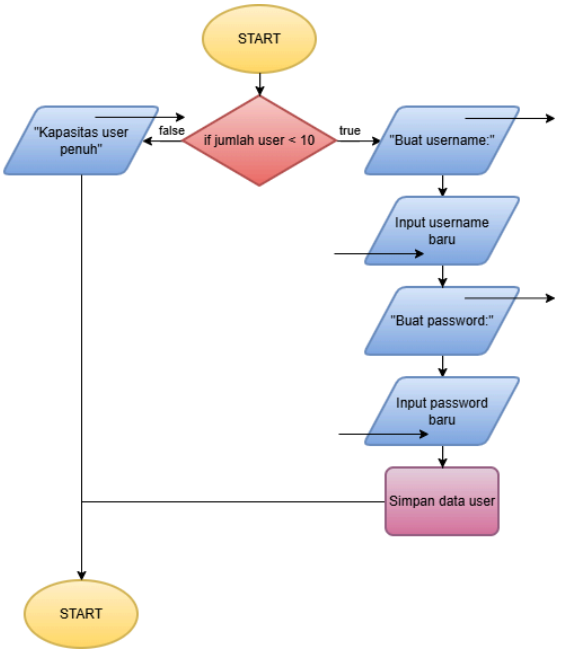
1. Flowchart

1.1 Main



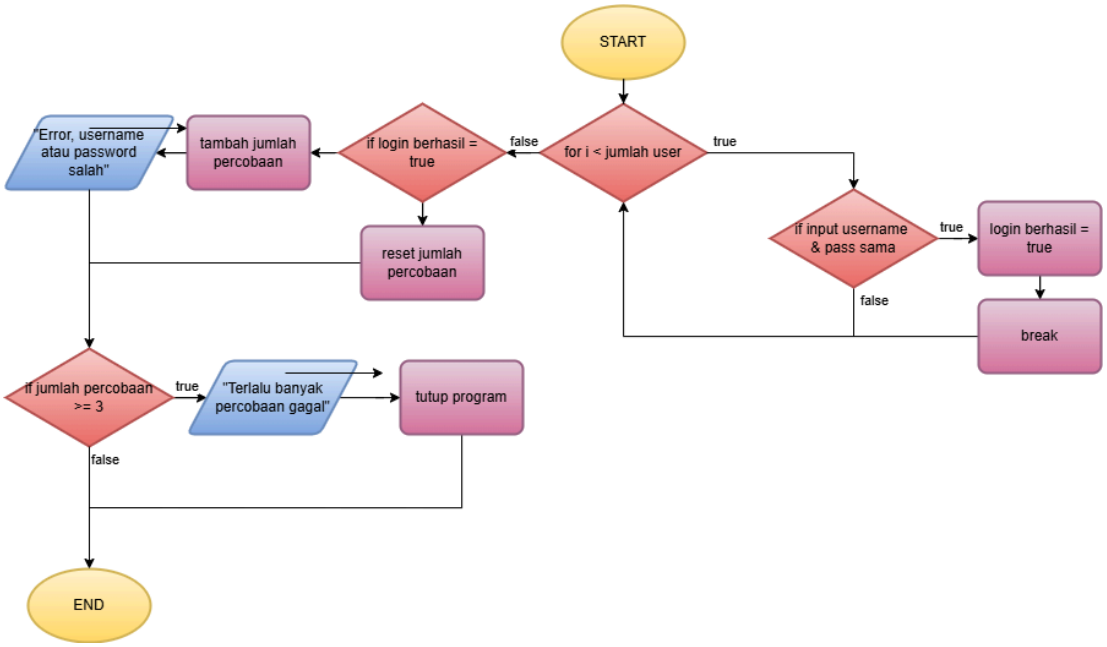
Gambar 1.1 Flowchart Main

1.2 Register



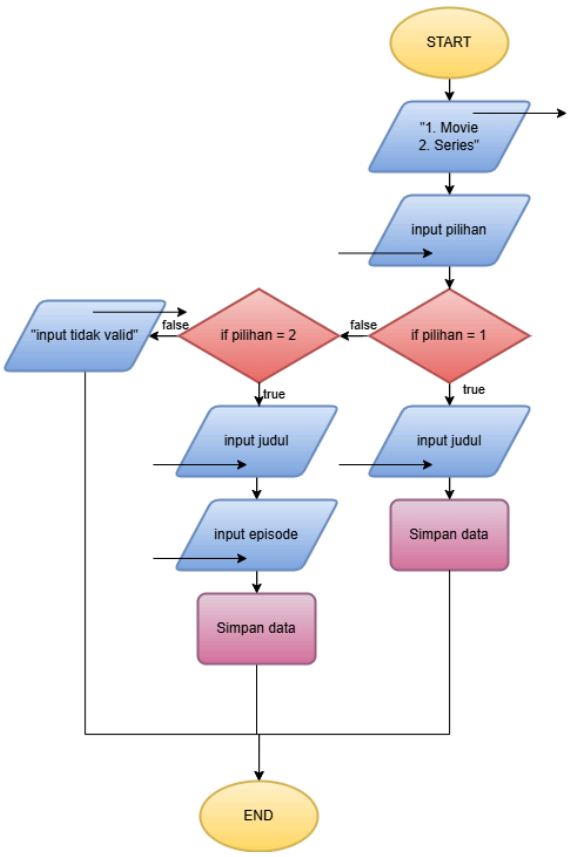
Gambar 1.2 Flowchart Register

1.3 Login



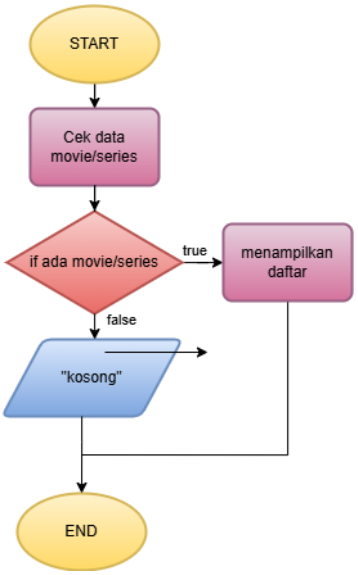
Gambar 1.3 Flowchart Login

1.4 Tambah Tontonan



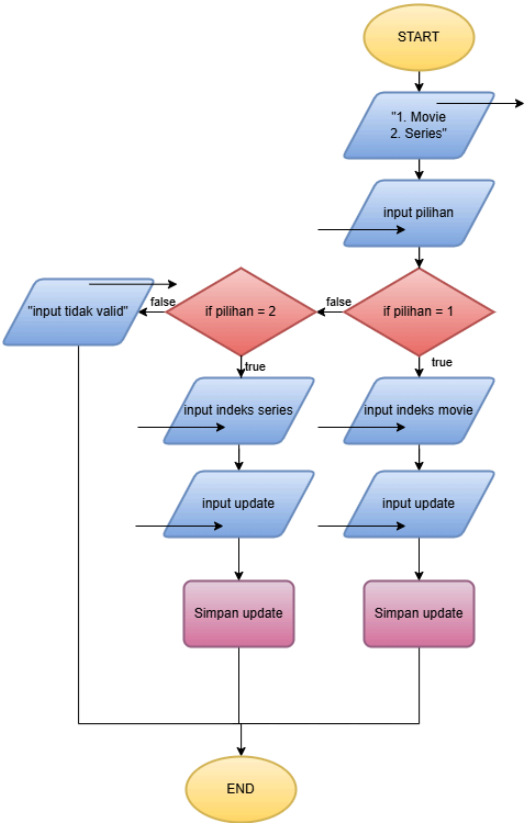
Gambar 1.4 Flowchart Tambah Tontonan

1.5 Lihat Daftar



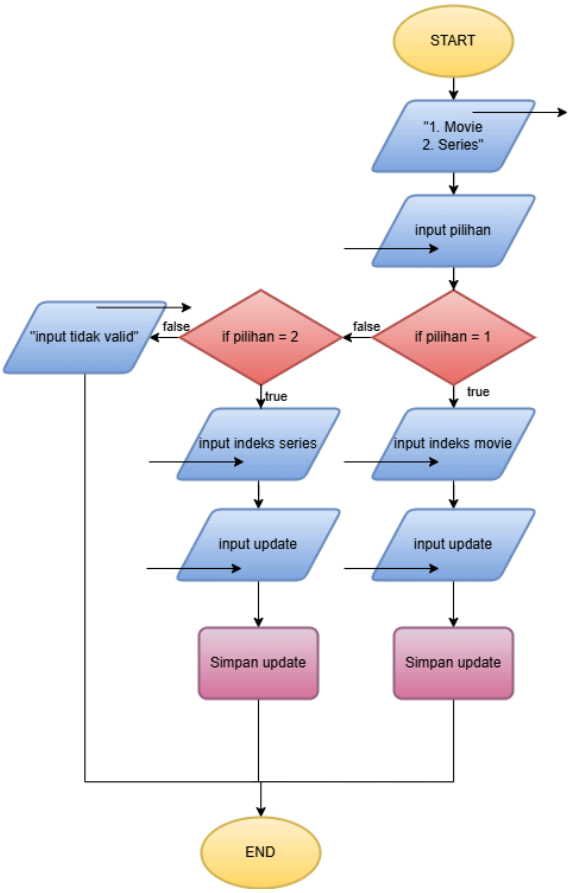
Gambar 1.5 Flowchart Lihat Daftar

1.6 Update Progress



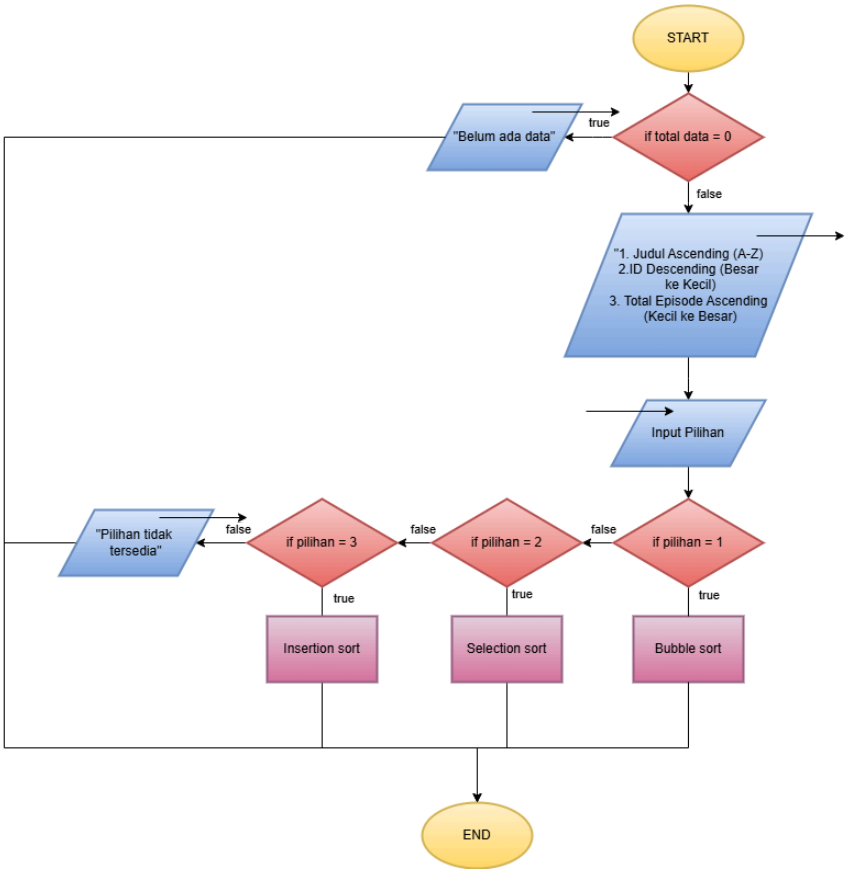
Gambar 1.6 Flowchart Update Progress

1.7 Hapus Data



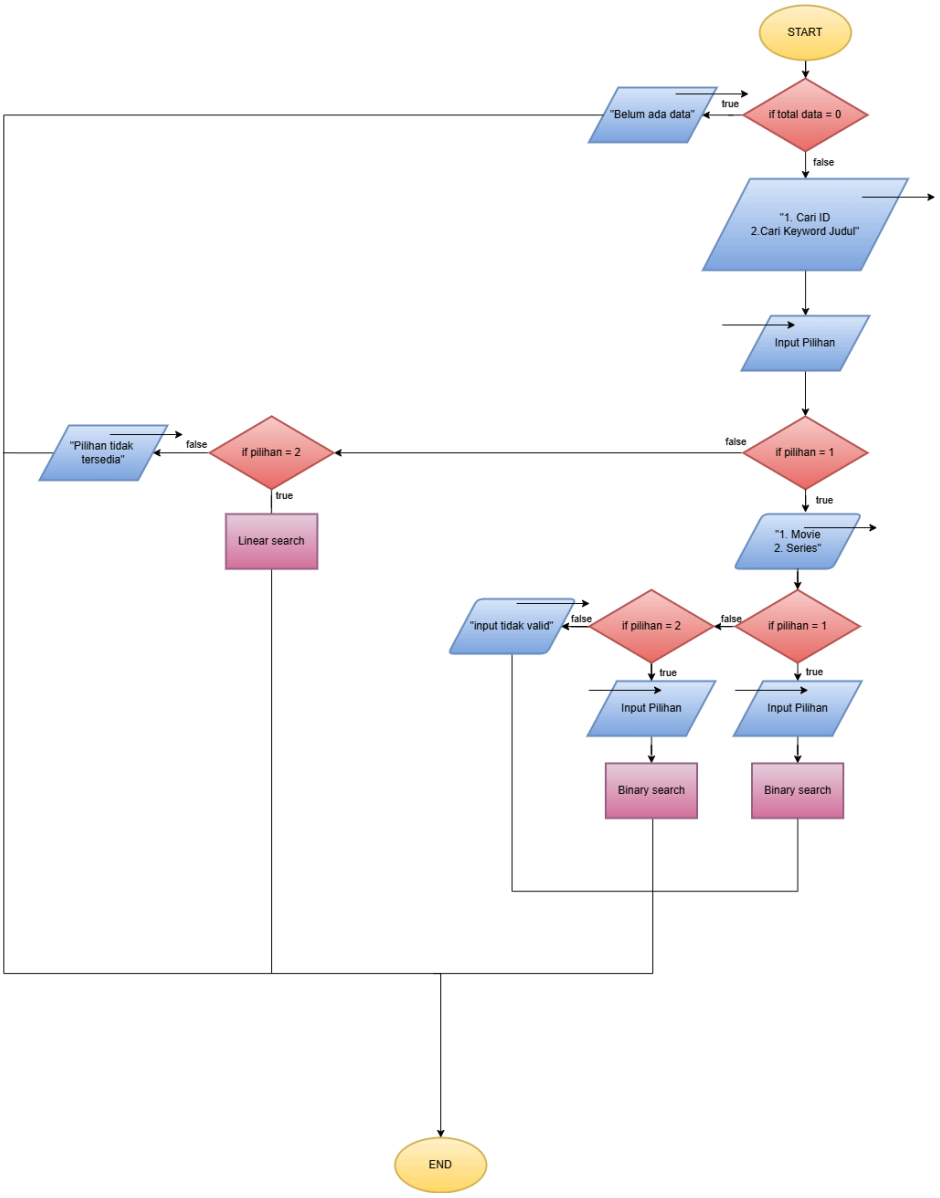
Gambar 1.7 Flowchart Hapus Data

1.8 Menu Sorting



Gambar 1.7 Flowchart Menu Sorting

1.9 Menu Searching



Gambar 1.7 Flowchart Menu Searching

2. Deskripsi Singkat Program

Program ini adalah aplikasi berbasis konsol yang berfungsi untuk mencatat dan memantau daftar progres tontonan (Movie dan Series). Program dilengkapi dengan sistem keamanan dan registrasi sederhana berupa fitur Login, di mana pengguna wajib memasukkan username dan password yang benar dengan batas maksimal tiga kali percobaan. Jika login gagal hingga batas percobaan tersebut, program akan otomatis berhenti.

Setelah berhasil masuk, pengguna disuguhkan Menu Utama yang memungkinkan mereka memilih opsi untuk menambah, melihat daftar yang terpisah, memperbarui progress, hingga menghapus data tontonan. Program akan terus berjalan menggunakan perulangan hingga pengguna memilih opsi untuk keluar atau logout. Aplikasi ini dirancang untuk memudahkan manajemen daftar hiburan secara cepat, privat, dan interaktif dalam satu sesi penggunaan.

3. Source Code

3.1 Header, Library, dan Deklarasi Struktur Data (Struct)

```
#include <iostream>
#include <string>
#include <limits>
#include <iomanip>
#include <fstream>
#include <cctype>
#include <stdexcept>
#include "ui-library.h"

using namespace std;

const string FILE_NAME = "database_movietrack.txt";

struct User {
    string username;
    string password;
    int movie_nextid;
    int series_nextid;
};

struct Tontonan {
    int id;
    string judul;
    string tipe;
    int total_eps;
    int eps_ditonton;
    string status;
    User* owner_ptr;
};
```

3.2 Fungsi Utilitas dan Manajemen File (Database)

```
void swap_tontonan(Tontonan* a, Tontonan* b) {
    Tontonan temp = *a;
    *a = *b;
    *b = temp;
}

int get_valid_int() {
    int val;
    cin >> val;
    if (cin.fail()) {
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        throw invalid_argument("Input tidak valid! Harap masukkan
angka.");
    }
    return val;
}

void simpan_data(User* daftar_user, int jumlah_user, Tontonan* daftar,
int jumlah_data) {
    try {
        ofstream file(FILE_NAME);
        if (!file.is_open()) {
            throw runtime_error("Gagal membuka file untuk menyimpan
data!");
        }

        file << jumlah_user << "\n";
        for (int i = 0; i < jumlah_user; i++) {
            file << daftar_user[i].username << "\n"
                 << daftar_user[i].password << "\n"
                 << daftar_user[i].movie_nextid << "\n"
                 << daftar_user[i].series_nextid << "\n";
        }

        file << jumlah_data << "\n";
        for (int i = 0; i < jumlah_data; i++) {
            file << daftar[i].id << "\n"
                 << daftar[i].judul << "\n"
                 << daftar[i].tipe << "\n"
                 << daftar[i].total_eps << "\n"
                 << daftar[i].eps_ditonton << "\n"
                 << daftar[i].status << "\n"
                 << daftar[i].owner_ptr->username << "\n";
```

```
        }
        file.close();
    } catch (const exception& e) {
        print_error(e.what());
    }
}

void muat_data(User* daftar_user, int* jumlah_user, Tontonan* daftar,
int* jumlah_data) {
    try {
        ifstream file(FILE_NAME);
        if (!file.is_open()) {
            throw runtime_error("Database belum ada atau gagal dibuka.
Membuat database baru...");
        }

        file >> *jumlah_user; file.ignore();
        for (int i = 0; i < *jumlah_user; i++) {
            getline(file, daftar_user[i].username);
            getline(file, daftar_user[i].password);
            file >> daftar_user[i].movie_nextid;
            file >> daftar_user[i].series_nextid;
            file.ignore();
        }

        file >> *jumlah_data; file.ignore();
        for (int i = 0; i < *jumlah_data; i++) {
            file >> daftar[i].id; file.ignore();
            getline(file, daftar[i].judul);
            getline(file, daftar[i].tipe);
            file >> daftar[i].total_eps;
            file >> daftar[i].eps_ditonton; file.ignore();
            getline(file, daftar[i].status);

            string owner_username;
            getline(file, owner_username);

            for(int j = 0; j < *jumlah_user; j++) {
                if(daftar_user[j].username == owner_username) {
                    daftar[i].owner_ptr = &daftar_user[j];
                    break;
                }
            }
        }
        file.close();
    } catch (const exception& e) {
```

```
        cout << YELLOW << "[Info] " << e.what() << RESET << "\n";
    }
}
```

3.3 Algoritma Sorting dan Searching

```
void bubbleSort_JudulAsc(Tontonan* daftar, int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (daftar[j].judul > daftar[j + 1].judul)
swap_tontonan(&daftar[j], &daftar[j + 1]);
        }
    }
}

void selectionSort_IdDesc(Tontonan* daftar, int n) {
    for (int i = 0; i < n - 1; i++) {
        int max_idx = i;
        for (int j = i + 1; j < n; j++) {
            if (daftar[j].id > daftar[max_idx].id) max_idx = j;
        }
        if (max_idx != i) swap_tontonan(&daftar[i], &daftar[max_idx]);
    }
}

void insertionSort_EpsAsc(Tontonan* daftar, int n) {
    for (int i = 1; i < n; i++) {
        Tontonan key = daftar[i];
        int j = i - 1;
        while (j >= 0 && daftar[j].total_eps > key.total_eps) {
            daftar[j + 1] = daftar[j];
            j = j - 1;
        }
        daftar[j + 1] = key;
    }
}

int binarySearch_ID(Tontonan* arrTemp[], int n, int idCari) {
    int kiri = 0;
    int kanan = n - 1;
    while (kiri <= kanan) {
        int tengah = kiri + (kanan - kiri) / 2;
        if (arrTemp[tengah]->id == idCari) return tengah;
        if (arrTemp[tengah]->id < idCari) kiri = tengah + 1;
        else kanan = tengah - 1;
    }
    return -1;
}

void linearSearch_Keyword(Tontonan* daftar, int jumlah_data, string
current_user, string keyword) {
    string keywordLower = keyword;
```

```
    for (char &c : keywordLower) c = tolower(c);

    bool ditemukan = false;
    cout << YELLOW << "\n>> HASIL PENCARIAN (Metode: Linear Search) <<" <<
RESET << "\n";
    cout << left << setw(5) << "ID" << setw(10) << "Tipe" << setw(45) <<
"Judul" << "Status\n";

    for (int i = 0; i < jumlah_data; i++) {
        if (daftar[i].owner_ptr->username == current_user) {
            string judulLower = daftar[i].judul;
            for (char &c : judulLower) c = tolower(c);

            if (judulLower.find(keywordLower) != string::npos) {
                cout << left << setw(5) << daftar[i].id
                     << setw(10) << daftar[i].tipe
                     << setw(45) << daftar[i].judul
                     << daftar[i].status << endl;
                ditemukan = true;
            }
        }
    }
    if (!ditemukan) cout << "(Tidak ada tontonan yang memuat kata kunci '"
<< keyword << "')\n";
}
int cari_id(Tontonan* daftar, int jumlah, int target, string owner, string
tipe, int iter = 0) {
    if (iter >= jumlah) return -1;
    if (daftar[iter].id == target && daftar[iter].owner_ptr->username ==
owner && daftar[iter].tipe == tipe) {
        return iter;
    }
    return cari_id(daftar, jumlah, target, owner, tipe, iter + 1);
}
```

3.4 Fitur Autentikasi (User Management)

```
void registrasi_akun(User* daftar_user, int* jumlah_user) {
    try {
        if (*jumlah_user >= 10) throw overflow_error("Kapasitas user
penuh!");
        cout << "Buat Username: "; cin >>
daftar_user[*jumlah_user].username;
        cout << "Buat Password: "; cin >>
daftar_user[*jumlah_user].password;
        daftar_user[*jumlah_user].movie_nextid = 1;
        daftar_user[*jumlah_user].series_nextid = 1;
        (*jumlah_user)++;
        print_success("Registrasi Berhasil!");
    } catch (const exception& e) {
        print_error(e.what());
    }
}

void proses_login(User* daftar_user, int jumlah_user, string user_in,
string pass_in, bool* login_berhasil, int* index_userlogin, int*
jumlah_percobaan, bool* program_berjalan) {
    for (int i = 0; i < jumlah_user; i++) {
        if (user_in == daftar_user[i].username && pass_in ==
daftar_user[i].password) {
            *login_berhasil = true;
            *index_userlogin = i;
            break;
        }
    }

    if (*login_berhasil) {
        *jumlah_percobaan = 0;
        loading_animation("Memverifikasi kredensial");
        print_success("Login Berhasil!");
    } else {
        (*jumlah_percobaan)++;
        cout << RED << "[Error] Username atau Password salah! Sisa
percobaan: " << 3 - *jumlah_percobaan << RESET << "\n";
        if (*jumlah_percobaan >= 3) {
            print_error("Terlalu banyak percobaan gagal. Program
otomatis keluar.");
            *program_berjalan = false;
        }
    }
}
```

3.5 Fitur Utama Program (CRUD Data Tontonan)

```
void tambah_tontonan(Tontonan* daftar, int* jumlah_data, User* user_aktif) {
    try {
        if (*jumlah_data >= 100) throw overflow_error("Maaf, kapasitas
daftar sudah penuh!");

        cout << "\nTipe (1. Movie, 2. Series, 0. Batal): ";
        int pt = get_valid_int();
        if (pt < 0 || pt > 2) throw out_of_range("Pilihan tipe tidak
tersedia.");
        if (pt == 0) return;

        cout << "Judul: "; cin.ignore(); getline(cin,
daftar[*jumlah_data].judul);
        if (pt == 1) {
            daftar[*jumlah_data].tipe = "Movie";
            daftar[*jumlah_data].id = user_aktif->movie_nextid++;
            daftar[*jumlah_data].total_eps = 1;
        } else {
            daftar[*jumlah_data].tipe = "Series";
            daftar[*jumlah_data].id = user_aktif->series_nextid++;
            cout << "Total Episode: ";
            int eps = get_valid_int();
            if (eps <= 0) {
                user_aktif->series_nextid--;
                throw invalid_argument("Jumlah episode harus lebih dari
0.");
            }
            daftar[*jumlah_data].total_eps = eps;
        }
        daftar[*jumlah_data].eps_ditonton = 0;
        daftar[*jumlah_data].status = "Rencana Menonton";
        daftar[*jumlah_data].owner_ptr = user_aktif;
        (*jumlah_data)++;
        print_success("Data berhasil disimpan!");
    } catch (const exception& e) {
        print_error(e.what());
    }
}

void lihat_daftar(Tontonan* daftar, int jumlah_data, string current_user) {
    clear_screen();
    cout << BLUE << BOLD << "\n>> DAFTAR MOVIE <<" << RESET << "\n";
    cout << left << setw(5) << "ID" << setw(55) << "Judul" << "Status\n";

    bool ada_movie = false;
    for (int i = 0; i < jumlah_data; i++) {
```

```
        if (daftar[i].owner_ptr->username == current_user && daftar[i].tipe
== "Movie") {
            cout << left << setw(5) << daftar[i].id
                 << setw(55) << daftar[i].judul
                 << daftar[i].status << endl;
            ada_movie = true;
        }
    }
    if(!ada_movie) cout << "(Kosong)\n";

    cout << BLUE << BOLD << "\n>> DAFTAR SERIES <<" << RESET << "\n";
    cout << left << setw(5) << "ID" << setw(55) << "Judul" << setw(15) <<
"Progress" << "Status\n";

    bool ada_series = false;
    for (int i = 0; i < jumlah_data; i++) {
        if (daftar[i].owner_ptr->username == current_user && daftar[i].tipe
== "Series") {
            string progress = to_string(daftar[i].eps_ditonton) + "/" +
to_string(daftar[i].total_eps);
            cout << left << setw(5) << daftar[i].id
                 << setw(55) << daftar[i].judul
                 << setw(15) << progress
                 << daftar[i].status << endl;
            ada_series = true;
        }
    }
    if(!ada_series) cout << "(Kosong)\n";
}

void update_progress(Tontonan* daftar, int jumlah_data, string current_user)
{
    try {
        cout << "\nUpdate Apa? (1. Movie, 2. Series, 0. Batal): ";
        int up = get_valid_int();
        if (up < 0 || up > 2) throw out_of_range("Pilihan tidak valid.");
        if (up == 0) return;

        string tipe_cari = (up == 1) ? "Movie" : "Series";
        cout << "\nMasukkan ID " << tipe_cari << " (0 Batal): ";
        int idC = get_valid_int();
        if (idC == 0) return;

        int indeks = cari_id(daftar, jumlah_data, idC, current_user,
tipe_cari);
        if (indeks == -1) throw runtime_error("ID tidak ditemukan di daftar
" + tipe_cari + " Anda.");
```

```
        if (tipe_cari == "Movie") {
            daftar[indeks].eps_ditonton = 1; daftar[indeks].status =
"Selesai";
            print_success("Berhasil diupdate menjadi Selesai!");
        } else {
            cout << "Episode terakhir ditonton: ";
            int eb = get_valid_int();
            if (eb < 0 || eb > daftar[indeks].total_eps) throw
invalid_argument("Episode tidak valid! Harus antara 0 - " +
to_string(daftar[indeks].total_eps));

            daftar[indeks].eps_ditonton = eb;
            daftar[indeks].status = (eb == daftar[indeks].total_eps) ?
"Selesai" : "Sedang Ditonton";
            print_success("Berhasil diupdate!");
        }
    } catch (const exception& e) {
        print_error(e.what());
    }
}

void hapus_tontonan(Tontonan* daftar, int* jumlah_data, string current_user)
{
    try {
        cout << "\nTipe yang dihapus (1. Movie, 2. Series, 0. Batal): ";
        int th = get_valid_int();
        if (th < 0 || th > 2) throw out_of_range("Pilihan tidak valid.");
        if (th == 0) return;

        string tipe_hapus = (th == 1) ? "Movie" : "Series";
        cout << "\nMasukkan ID yang ingin dihapus (0 Batal): ";
        int idH = get_valid_int();
        if (idH == 0) return;

        int indeks_hapus = cari_id(daftar, *jumlah_data, idH, current_user,
tipe_hapus);
        if (indeks_hapus == -1) throw runtime_error("ID tidak ditemukan.");

        char yakin; cout << "Yakin hapus '" << daftar[indeks_hapus].judul <<
"'? (y/n): "; cin >> yakin;
        if (tolower(yakin) == 'y') {
            for (int j = indeks_hapus; j < *jumlah_data - 1; j++) {
                daftar[j] = daftar[j + 1];
            }
            (*jumlah_data)--;
            print_success("Data berhasil dihapus!");
        }
    } catch (const exception& e) {
```

```
        print_error(e.what());
    }
}
```

3.6 Fungsi Utama (int main)

```
int main() {
    Tontonan daftar[100];
    int jumlah_data = 0;
    User daftar_user[10];
    int jumlah_user = 0;
    int jumlah_percobaan = 0;
    bool program_berjalan = true;

    muat_data(daftar_user, &jumlah_user, daftar, &jumlah_data);

    while (program_berjalan) {
        string user_in, pass_in;
        bool login_berhasil = false;
        int index_userlogin = -1;

        cetak_garis();
        cout << "\n" << GREEN << BOLD << "   SELAMAT DATANG DI MOVIE-TRACK
" << RESET;
        cetak_garis();
        cout << "\n(Ketik 'register' untuk buat akun)";
        cout << "\nUsername: "; cin >> user_in;

        if (user_in == "register") {
            registrasi_akun(daftar_user, &jumlah_user);
            simpan_data(daftar_user, jumlah_user, daftar, jumlah_data);
            continue;
        } else {
            cout << "Password: "; cin >> pass_in;
            proses_login(daftar_user, jumlah_user, user_in, pass_in,
&login_berhasil, &index_userlogin, &jumlah_percobaan, &program_berjalan);
            if (!login_berhasil) continue;
        }

        if (login_berhasil) {
            int pilihan = -1;
            string current_user = daftar_user[index_userlogin].username;
            string judul_menu = "MENU UTAMA (User: " + current_user + ")";

            do {
                try {
                    cetak_garis(judul_menu);
                    cout << "\n1. Tambah Tontonan";
                    cout << "\n2. Lihat Daftar";
                    cout << "\n3. Update Progress";
                    cout << "\n4. Hapus Data";
                    cout << "\n5. LOGOUT";
```

```
                    cout << "\n6. KELUAR PROGRAM";
                    cout << "\nPilih menu: ";

                    pilihan = get_valid_int();

                    if (pilihan == 1) {
                        tambah_tontonan(daftar, &jumlah_data,
&daftar_user[index_userlogin]);
                        simpan_data(daftar_user, jumlah_user, daftar,
jumlah_data);
                    }
                    else if (pilihan == 2) lihat_daftar(daftar, jumlah_data,
current_user);
                    else if (pilihan == 3) {
                        update_progress(daftar, jumlah_data, current_user);
                        simpan_data(daftar_user, jumlah_user, daftar,
jumlah_data);
                    }
                    else if (pilihan == 4) {
                        hapus_tontonan(daftar, &jumlah_data, current_user);
                        simpan_data(daftar_user, jumlah_user, daftar,
jumlah_data);
                    }
                    else if (pilihan == 5) {
                        char yakin; cout << "Yakin Logout? (y/n): "; cin >>
yakin;
                        if (tolower(yakin) == 'y') {
                            loading_animation("Logout");
                            clear_screen();
                        } else {
                            pilihan = 0;
                        }
                    }
                    else if (pilihan == 6) {
                        char yakin; cout << "Yakin Keluar Program? (y/n): ";
cin >> yakin;
                        if (tolower(yakin) == 'y') {
                            program_berjalan = false;
                            simpan_data(daftar_user, jumlah_user, daftar,
jumlah_data);
                            cout << GREEN << "Terima kasih telah menggunakan
Movie-Track!\n" << RESET;
                        } else pilihan = 0;
                    }
                    else throw out_of_range("Pilihan menu tidak tersedia!");

                } catch (const exception& e) {
                    print_error(e.what());
```

```
                }

            } while (pilihan != 5 && pilihan != 6);
        }
    }
    return 0;
}
```

4. Hasil Output

```
=====================================
   SELAMAT DATANG DI MOVIE-TRACK
=====================================
(Ketik 'register' untuk buat akun)
Username: register
Buat Username: yar
Buat Password: 043
Registrasi Berhasil!
```

Gambar 4.1 Register

```
=====================================
   SELAMAT DATANG DI MOVIE-TRACK
=====================================
(Ketik 'register' untuk buat akun)
Username: yar
Password: 043
```

Gambar 4.2 Log In

```
=== MENU UTAMA (User: yaralyar) ===
1. Tambah Tontonan
2. Lihat Daftar
3. Update Progress
4. Hapus Data
5. Urutkan Tontonan
6. Cari Tontonan
7. LOGOUT
8. KELUAR PROGRAM
Pilih menu:
```

Gambar 4.3 Menu Utama

```
=== MENU UTAMA (User: yaralyar) ===
1. Tambah Tontonan
2. Lihat Daftar (Terpisah)
3. Update Progress
4. Hapus Data
5. Urutkan Tontonan (Sorting)
6. LOGOUT
7. KELUAR PROGRAM
Pilih menu: 1

Tipe (1. Movie, 2. Series, 0. Batal): 1
Judul: Dead Poet Society
Data berhasil disimpan!
```

Gambar 4.4 Menu 1

```
>> DAFTAR MOVIE <<
ID   Judul                                                Status
5    Avatar: Fire and Ash                                 Rencana Menonton
4    Even if This Love Disappear From the World Tonight   Rencana Menonton
1    Interstellar                                         Rencana Menonton
2    Jojo Rabbit                                          Rencana Menonton
6    Na Willa                                             Rencana Menonton
3    Sonny Boy                                            Rencana Menonton
7    Dead Poet Society                                    Rencana Menonton

>> DAFTAR SERIES <<
ID   Judul                                                Progress      Status
1    Anohimita Hana no Namae wa Bokutachi wa Mada Shiranai 0/12         Rencana Menonton
3    Fumetsu no Anata e                                   0/12          Rencana Menonton
2    Mahou Shoujo Madoka Magica                           0/12          Rencana Menonton
4    Shigatsu wa Kimi no Uso                              0/12          Rencana Menonton

=== MENU UTAMA (User: yaralyar) ===
1. Tambah Tontonan
2. Lihat Daftar (Terpisah)
3. Update Progress
4. Hapus Data
5. Urutkan Tontonan (Sorting)
6. LOGOUT
7. KELUAR PROGRAM
Pilih menu:
```

Gambar 4.5 Menu 2

```
=== MENU UTAMA (User: yaralyar) ===
1. Tambah Tontonan
2. Lihat Daftar (Terpisah)
3. Update Progress
4. Hapus Data
5. Urutkan Tontonan (Sorting)
6. LOGOUT
7. KELUAR PROGRAM
Pilih menu: 3

Update Apa? (1. Movie, 2. Series, 0. Batal): 1

--- Daftar Movie Anda ---
ID      Judul
5       Avatar: Fire and Ash
4       Even if This Love Disappear From the World Tonight
1       Interstellar
2       Jojo Rabbit
6       Na Willa
3       Sonny Boy
7       Dead Poet Society

Masukkan ID (0 Batal): 4
Berhasil diupdate menjadi Selesai!
```

Gambar 4.6 Menu 3

```
=== MENU UTAMA (User: yaralyar) ===
1. Tambah Tontonan
2. Lihat Daftar (Terpisah)
3. Update Progress
4. Hapus Data
5. Urutkan Tontonan (Sorting)
6. LOGOUT
7. KELUAR PROGRAM
Pilih menu: 4

Tipe yang dihapus (1. Movie, 2. Series, 0. Batal): 1

Masukkan ID yang ingin dihapus (0 Batal): 3
Yakin hapus 'Sonny Boy'? (y/n): y
Data berhasil dihapus!
```

Gambar 4.7 Menu 4

```
=== MENU UTAMA (User: yaralyar) ===
1. Tambah Tontonan
2. Lihat Daftar (Terpisah)
3. Update Progress
4. Hapus Data
5. Urutkan Tontonan (Sorting)
6. LOGOUT
7. KELUAR PROGRAM
Pilih menu: 5

=== MENU SORTING ===
1. Judul Ascending (A-Z) [Bubble Sort]
2. ID Descending (Besar ke Kecil) [Selection Sort]
3. Total Episode Ascending (Kecil ke Besar) [Insertion Sort]
0. Batal
Pilih metode sorting:
```

Gambar 4.8 Menu 5

```
=== MENU SEARCHING ===
1. Cari ID (Menggunakan Binary Search)
2. Cari Keyword Judul (Menggunakan Linear Search)
0. Batal
Pilih metode pencarian: 1

Tipe yang dicari (1. Movie, 2. Series): 1
Masukkan ID: 2

>> DATA DITEMUKAN <<
ID      : 2
Judul   : Na Willa
Tipe    : Movie
Status  : Rencana Menonton
```

Gambar 4.8 Menu 6

```
=== MENU SEARCHING ===
1. Cari ID (Menggunakan Binary Search)
2. Cari Keyword Judul (Menggunakan Linear Search)
0. Batal
Pilih metode pencarian: 2
Masukkan Keyword Judul: Project

>> HASIL PENCARIAN (Metode: Linear Search) <<
ID   Tipe      Judul                                    Status
5    Movie     Project Hail Mary                        Rencana Menonton
```

Gambar 4.8 Menu 7

```
=== MENU UTAMA (User: yaralyar) ===
1. Tambah Tontonan
2. Lihat Daftar (Terpisah)
3. Update Progress
4. Hapus Data
5. Urutkan Tontonan (Sorting)
6. LOGOUT
7. KELUAR PROGRAM
Pilih menu: 6
Yakin Logout? (y/n): y

===================================
   SELAMAT DATANG DI MOVIE-TRACK
===================================
(Ketik 'register' untuk buat akun)
Username:
```

Gambar 4.9 Logout

```
=== MENU UTAMA (User: yaralyar) ===
1. Tambah Tontonan
2. Lihat Daftar (Terpisah)
3. Update Progress
4. Hapus Data
5. Urutkan Tontonan (Sorting)
6. LOGOUT
7. KELUAR PROGRAM
Pilih menu: 7
Yakin Keluar Program? (y/n): y
PS C:\Praktikum-APL>
```

Gambar 4.10 Tutup Program

5. Langkah-langkah GIT

5.1 GIT Add

```
PS C:\Praktikum-APL\Post-test> git add .
```

Gambar 5.1 GIT Add

Perintah git add . digunakan untuk memasukkan semua perubahan file di direktori saat ini ke dalam *staging area*. Proses ini merupakan tahap persiapan sebelum perubahan tersebut direkam secara permanen ke dalam *repository* menggunakan perintah *commit*.

5.2 GIT Commit

```
PS C:\Praktikum-APL> git commit -m "posttest 6"
[main 87b7d38] posttest 6
 3 files changed, 610 insertions(+)
 create mode 100644 Post-test/Post-test-APL-6/2509106043-AlyaKamilaFauziyah-PT-6.cpp
 create mode 100644 Post-test/Post-test-APL-6/2509106043-AlyaKamilaFauziyah-PT-6.exe
 create mode 100644 Post-test/Post-test-APL-6/database_movietrack.txt
```

Gambar 5.2 GIT Commit

Perintah git commit -m berfungsi untuk menyimpan semua perubahan dari *staging area* secara permanen ke dalam *repository* lokal. Opsi -m digunakan untuk menyertakan pesan deskriptif singkat secara langsung yang menjelaskan perubahan apa saja yang telah dilakukan pada versi tersebut.

5.3 GIT Push

```
PS C:\Praktikum-APL> git push
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 16 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (7/7), 107.06 KiB | 6.69 MiB/s, done.
Total 7 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/yaralyar07/praktikum-apl.git
   2bc9a83..87b7d38  main -> main
PS C:\Praktikum-APL>
```

Gambar 5.3 GIT Push

Perintah git push berfungsi untuk mengunggah semua perubahan yang sudah di-*commit* dari *repository* lokal ke *repository* jarak jauh (*remote*), seperti GitHub. Langkah ini memastikan versi terbaru dari kodemu tersimpan secara *online* sehingga aman dan dapat diakses dari perangkat lain.